Expense Tracker + Plaid — Product Spec (Living Document)

1) Product Purpose

Build a **production-ready personal finance tracker** that helps users:

- **Understand spending** (where money goes, trends, categories)
- **Stay in control** (budgets, alerts, planned expenses)
- **Reduce manual work** (bank sync with Plaid, smart categorization)
- **Make better decisions** (insights, forecasting, “what changed?” explanations)

Primary success metric: users can reliably see **accurate monthly totals** and **trust the data**, with minimal maintenance.

---

2) Target Users & Jobs-To-Be-Done

Primary user

A person who wants a simple, trustworthy view of spending and cash flow, without needing Excel.

Key jobs

- “Show me **how much I spent this month** and on what.”
- “Track **income vs expenses** and net change.”
- “Keep everything accurate even if banks show pending/changes.”
- “Make it easy to find transactions and fix categories.”

---

3) Core Principles (Non-negotiables)

1. **Data correctness > fancy UI**
2. **Idempotent syncing** (no duplicates, safe to retry)
3. **Security & privacy first** (tokens server-only, RLS enforced)
4. **Explainability** (“why did my totals change?”)
5. **Progressive enhancement** (manual-first works, Plaid adds power)

---

4) Feature Set Overview

Must-Have (MVP, production-grade baseline)

1. **Authentication**
    - Email/password via Supabase Auth
    - Verified user sessions
2. **Accounts**
    - Manual accounts (cash, checking, credit card)
    - Plaid-linked accounts (later in MVP+)
3. **Categories**
    - User-owned categories with type: expense/income
    - Default category set on first run
4. **Transactions**
    - CRUD for manual transactions
    - Store signed amounts in DB: **expense negative**, **income positive**
    - UI displays expense as positive (presentation-only)
5. **Dashboard**
    - Monthly totals: spending, income, net
    - Basic category breakdown
6. **Search & Filters**
    - By date range, account, category, text query
7. **RLS + Authorization correctness**
    - User can only access their own rows
    - Insert/update checks ensure referenced account/category belongs to user

MVP+ (Plaid integration)

8. **Connect bank (Plaid Link)**
    - Link token creation (server)
    - Link UI (client)
    - Public token exchange (server)
    - Store institution info + item status
9. **Account import from Plaid**
    - Upsert accounts with plaid ids, mask, subtype, etc.
10. **Transaction sync**

- Initial import (~90 days)
- Incremental sync via cursor
- Handle `added`, `modified`, `removed`
- Pending vs posted handling

11. **Sync UX**

- Show last sync time/status per bank connection
- Manual “Sync now” button

Differentiators (what makes it stand out)

12. **Smart categorization system**
- Rules engine (merchant → category)
- “Learn from my edits”
- Raw Plaid categories stored for remapping
13. **Budgets & alerts**
- Monthly budget per category
- “You’re 80% through your dining budget”
14. **Anomaly & change detection**
- “Spending increased by $X due to Y merchants”
15. **Recurring transactions**
- Detect subscriptions & recurring bills
- Forecast upcoming charges
16. **Financial health view**
- Trends, savings rate, discretionary vs fixed spending
17. **Export / portability**
- CSV export for date range or full history

---

5) How It Should Work (User Journeys)

Journey A: New user (manual-first)

1. User signs up/logs in
2. App creates default categories (or prompts)
3. User creates a manual account (Cash/Checking)
4. User adds transactions
5. Dashboard shows monthly totals

Definition of done: user sees correct totals and can filter/search without errors.

Journey B: Connect bank (Plaid)

1. User goes to “Banks / Connections”
2. Click “Connect bank”
3. Client requests **link_token** from server
4. Plaid Link opens; user connects institution
5. Client receives **public_token + metadata**
6. Server exchanges public token for **access_token + plaid_item_id**
7. Server stores:
    - `plaid_items` row (institution info, status, cursor empty, last sync fields)
    - `plaid_item_secrets` row (encrypted access token)

8. Server fetches accounts and upserts into `accounts`

Definition of done: connected institution is visible and accounts appear.

Journey C: Initial transactions import (90 days)

1. After item is stored, server calls transactions sync
2. For each returned transaction:
    - Upsert by `plaid_transaction_id`
    - Map to internal `account_id` (via plaid account id → accounts table)
    - Store signed amount (apply your "expense negative" convention)
    - Store `pending`, `authorized_at`, `name`, `payment_channel`, `raw_category`
3. Store cursor after successful write
4. Update `last_sync_at/status`

Definition of done: re-running import does not create duplicates.

Journey D: Incremental sync (daily / on-demand)

1. User triggers sync or scheduled sync runs
2. Server calls Plaid sync with stored cursor
3. Apply changes:
    - `added`: insert/upsert
    - `modified`: update relevant fields
    - `removed`: set `is_removed = true` (never hard delete)
4. Cursor advances only after successful commit

Definition of done: totals remain stable and reflect Plaid corrections.

---

6) Data & Money Semantics (Critical)

Amount sign convention (already decided)

- DB stores **signed** amounts:
    - Expenses: negative
    - Income: positive
- UI:
    - Expenses shown as positive for readability

    - Net = income + expenses (since expenses are negative)

Pending vs removed

- **Pending**: can change → update the same record (or link posted to pending if Plaid provides relation)
- **Removed**: must not count in totals → `is_removed = true`

Idempotency keys

- Plaid transaction identity: `plaid_transaction_id` must be **unique**
- Plaid account identity: `plaid_account_id` should be unique (nullable allowed)

---

7) System Components (Architecture)

Client (Next.js UI)

- Pages: Dashboard, Transactions, Accounts, Categories, Connections
- Connect flow uses Plaid Link client library
- Calls server route handlers for Plaid operations

Server (Next.js Route Handlers)

- `POST /api/plaid/create-link-token`
- `POST /api/plaid/exchange-public-token`
- `POST /api/plaid/sync-transactions`
- later: `POST /api/plaid/webhook`

Database (Supabase Postgres)

- User-owned: accounts, categories, transactions, plaid_items
- Server-only secret table: plaid_item_secrets

---

8) Security & Threat Model (Non-negotiables)

- **No access_token or secret** ever returned to client
- `plaid_item_secrets`:
    - RLS enabled
    - **no policies** (blocked from client)

- `plaid_items`, `accounts`, `transactions`, `categories`:
  - RLS enabled
  - policies enforce `user_id = auth.uid()`
- Transaction insert/update must enforce:
  - referenced account belongs to the same user
  - referenced category belongs to the same user (if present)

---

9) Observability & Debuggability

For each plaid item, store:

- `last_sync_at`
- `last_sync_status` (success/error)
- `last_sync_error` (short text or JSON)
- cursor string

Logs/telemetry (minimum)

- Sync start/end events with counts:
  - added/modified/removed
- Error logs with plaid error type/code (without leaking tokens)

---

10) Milestones (Implementation Plan)

Milestone 1 — Core app stable (manual)

- Auth done
- Accounts/Categories/Transactions CRUD
- Dashboard totals correct
- RLS hardened

Milestone 2 — Plaid vertical slice

- Link token endpoint
- Link UI connect button
- Exchange endpoint + store item + secret
- Account import upsert
- Initial transactions sync (90 days)
- Cursor storage + last sync status UI

Milestone 3 — Production polish

- Webhooks + background sync
- Better failure UX (reconnect, retry)
- Budgeting + rules engine foundation

Milestone 4 — Differentiators

- Smart rules
- Recurring detection
- Insights/anomaly explanations
- Export + shareable reports

---

11) Acceptance Criteria (for “Plaid connected” feature)

A user can:

- Connect a bank in sandbox
- See institution + accounts
- See imported transactions
- Re-run sync without duplicates
- Observe removed transactions excluded from totals
- See sync status and last sync time

---

12) Open Decisions (track here)

- CA vs US institutions (country_codes)
- Webhook strategy (tunnel in dev vs deploy early)
- How to map categories initially (Plaid → internal)
- Pending→posted linking policy

If you want, I can turn this into a stricter **PRD format** (Goals, Non-Goals, Requirements, Edge cases, API contracts, Data model), or into a **checklist you can tick off during development**.